

第2章：基本制御フロー

条件分岐・ループ・パターンマッチ

1 if / else if / else

他の言語と違い、条件式に括弧 () が不要。比較演算子 (==, !=, <, >, <=, >=) と論理演算子 (!, ||, &&) が使える。

比較演算子も演算の一種なので、**型が一致していないとコンパイルエラーになる**。例えば f64 の変数と整数リテラル 42 を比較しようとするエラーになるため、42.0 と書いて型を合わせる必要がある。

```
1 fn main() {
2     let x = 41.9999_f64;
3     if x < 42.0 {                // 42 ではなく 42.0 と書く
4         println!("42より小さい");
5     } else if x == 42.0 {
6         println!("42に等しい");
7     } else {
8         println!("42より大きい");
9     }
10 }
```

2 loop

無限ループ。break で抜け出す。条件を書き忘れると整数がオーバーフローしてパニックするので注意。

```
1 fn main() {
2     let mut x = 0;
3     loop {
4         x += 1;
5         if x == 42 {
6             break;
7         }
8     }
9     println!("{}", x); // 42
10 }
```

3 while

条件が true の間ループし続ける。条件が false になったら終了。出力がなくてもエラーではない (println! を書かなければ何も表示されないだけ)。

```

1 fn main() {
2     let mut x = 0;
3     while x != 42 {
4         x += 1;
5     }
6     println!("{}", x); // 42
7 }

```

4 for

イテレータを使って繰り返す。Rust では `..` を使うだけで整数のシーケンス（イテレータ）を簡単に作れる。

```

1 fn main() {
2     // 0..5 → 0, 1, 2, 3, 4（終端を含まない）
3     for x in 0..5 {
4         println!("{}", x);
5     }
6
7     // 0..=5 → 0, 1, 2, 3, 4, 5（終端を含む）
8     for x in 0..=5 {
9         println!("{}", x);
10    }
11
12    // 配列を直接回す
13    let nums = [10, 20, 30];
14    for n in nums {
15        println!("{}", n);
16    }
17 }

```

`..` と `..=` の違いは終端を含むかどうかだけ。Python の `range(0, 5)` が `0..5` に相当する。

5 match

値が何かによって分岐する。他言語の `switch` に相当するが、全パターンを網羅しないとコンパイルエラーになる点が大きな違い。

`match` の構造は「パターン => 処理」で、=> は矢印（不等号ではない）。

```

1 fn main() {
2     let x = 42;
3
4     match x {
5         0 => {
6             println!("ゼロ");
7         }
8     }
9 }

```

```

8      1 | 2 => {                                // | は「または」 (1か2のどちら
      か)
9          println!("1か2");
10     }
11     3..=9 => {                                // 範囲指定 (3以上9以下)
12         println!("3から9");
13     }
14     matched_num @ 10..=100 => { // @ でマッチした値に名前をつ
      ける
15         println!("{}", matched_num);
16     }
17     _ => {                                    // _ はワイルドカード (それ以外全
      部)
18         println!("それ以外");
19     }
20 }
21 }

```

1 | 2 は「1 または 2」という列挙。1..=2 (範囲) とは書き方が違うが、今回の場合は結果は同じ。

matched_num @ 10..=100 は「10 以上 100 以下にマッチして、その値を matched_num という名前で使う」という意味。

6 loop から値を返す

break に値を付けると、loop 全体がその値を返す式になる。

```

1 fn main() {
2     let mut x = 0;
3     let result = loop {
4         x += 1;
5         if x == 42 {
6             break x * 2; // break に値を付けて返す
7         }
8     };
9     println!("{}", result); // 84
10 }

```

7 ブロック式から値を返す

Rust では if、match、{ } ブロックはすべて値を返す**式**。最後の行にセミコロンを付けなければその値が返る (第 1 章の関数と同じルール)。

これを使うと、Python の三項演算子 (-1 if x < 42 else 1) と同じことが if 式で書ける。

```

1 fn main() {
2     let x = 42;
3

```

```

4 // if を三項演算子のように使う (Pythonの「A if 条件 else B」と
  同じ発想)
5 let label = if x < 42 { "小さい" } else { "大きい" };
6 println!("{}", label);
7
8 // ブロックも値を返せる
9 let msg = {
10     let y = x * 2;
11     format!("x={}, 2倍={}", x, y) // セミコロンなし → これが
    返り値
12 };
13 println!("{}", msg);
14 }

```

文と式の区別

セミコロンあり → 文 (値を返さない)。セミコロンなし → 式 (値を返す)。この区別が Rust 全体で一貫して使われる重要なルール。

まとめ

構文	説明
if x < 42 { }	条件分岐 (括弧不要)
loop { break; }	無限ループ + break
while x != 0 { }	条件付きループ
for x in 0..5 { }	イテレーターループ (終端含まず)
for x in 0..=5 { }	イテレーターループ (終端含む)
match x { _ => }	網羅的パターンマッチ
break x * 2	loop から値を返す
let v = if ... { } else { }	if 式 (三項演算子代わり)